

Python Assignment 7

1. What is the difference between local variables and global variables?

Local variables work inside a function or block and a global variable works for the whole program. A local variable can access a global variable's value but is destroyed at the end of the block and is not available thereafter.

```
value1 = 10          #Global variable

def demo1():
    value2 = 11      #Local variable

    print("inside function demo1 is : ", value1)    #10
    print("inside function demo1 is : ", value2)    #22

def demo2():
    print("inside function demo2 is : ", value1)    #10
    print("inside function demo2 is : ", value2)    #Error

demo1()
demo2()
```

In this example, we see that the global variable's value is accessible to all functions, but the `demo1` function's variable is not available for the `demo2` function. This is the difference between local and global variables.

2. Why should excessive use of global variables be avoided in large programs?

A **global variable** is a variable declared **outside any function**. It can be accessed from anywhere in the same module.

Because global variables can be accessed by all other functions, sometimes any function can modify a global variable, making it hard to determine where an incorrect value was assigned.

Different functions become dependent on the same global data. A change in one function may unexpectedly affect another.

Multiple functions modifying the same variable can produce unexpected behavior and bugs.

```
value = 10
def increment():
    global value
    value = value + 10    # except 20
def decrement():
    global value
    value = value - 5     #except 5
increment()
decrement()
print(value)             #but answer is 15
```

3. Explain the use of the global keyword. When should it be used?

A **global variable** is a variable declared **outside any function**. It can be accessed from anywhere in the same module.

According to the official Python documentation, the global keyword is used to tell Python that a variable inside a function refers to the global (module-level) variable, not a new local variable. Python assumes it is a local variable unless we explicitly declare it as global.

```
value = 10

def display():
    print(value)
#Here, count is only being read, so the global keyword is not required.

def increment():
    global value
    value = value + 10 # except 20

def decrement():
    global value
    value = value - 5 #except 5

def main():
    display()
    increment()
    decrement()

    print(value) #but answer is 15

if __name__ == "__main__":
    main()
```

4. What is the special variable `__name__` in Python? Who assigns its value?

`__name__` is a special built-in variable in Python that stores the name of the current module. Its value is **automatically assigned by the Python interpreter**. If a file is executed directly, `__name__` is set to `"__main__"`. If the file is imported as a module, `__name__` is set to the module's name. It is commonly used with `if __name__ == "__main__":` to execute code only when the file is run directly.

In python we can check..

```
def main():
    print(__name__)

if __name__ == "__main__":
    main()
```

5. What is the meaning of `__main__` in Python execution?

`__main__` indicates that the current Python file is the **entry point** of the program.

`__main__` is the top-level execution environment in Python. When a Python file is executed directly, the Python interpreter automatically assigns the special variable `__name__` the value `"__main__"`. Therefore, the code inside `if __name__ == "__main__":` executes only when the file is run directly. If the file is imported as a module into another program, `__name__` is assigned the module's name instead, and the code inside the `if` block is not executed.

File1.py

```
def file1():  
    print("i am from file1 function")  
  
print("i am print from file1")
```

file2.py

```
import A7Q5  
  
print("hello im from file2 mean (A7Q5(1))")
```

OUTPUT :

```
i am print from file1  
hello im from file2 mean (A7Q5(1))
```

here we see file2 run and read file1 content. Now we modify the file1 from using `__name__ == "__main__":`

Modify file1.py

```
def file1():  
    print("i am from file1 function")  
  
def main():  
    pass  
  
__name__ == "__main__":  
print("i am print from file1")
```

OUTPUT :

```
hello im from file2 mean (A7Q5(1))
```

Here we see file2 cannot read file1 content... we build file1 inside `__name__ == "__main__":`. Now check file validation if condition not true then cannot enter inside.

`__name__ == "__main__"` acts like a gatekeeper.

If the file is run directly → block executes.

If imported → block is skipped.

This prevents unwanted side effects (like random print statements) when using files as modules.

6. Can a function return another function? Explain conceptually.

Yes, When a function returns another function, it returns **the function object itself**, not the result of calling that function.

```
def function1():
    def function2():
        print("I am from function2")
    return function2

def main():
    Ret = function1()
    Ret()

if __name__ == "__main__":
    main()
```

7. What is a module in Python? How is it different from a function?

A **module** in Python is a file with a .py extension that contains reusable code such as variables, functions, classes, and executable statements. Modules help organize code and allow it to be reused in other programs using the import statement. A **function**, on the other hand, is a named block of code that performs a specific task and is defined using the def keyword. A module can contain multiple functions, whereas a function performs one particular operation.

File1 :A7Q7.py

```
PI = 3.14159

def add(a, b):          #Function
    return a + b

def subtract(a, b):     #Function
    return a - b
```

File2 :A7Q7(1).py

```
#Module
import A7Q7

print(A7Q7.add(10, 20))
print(A7Q7.PI)
```

OUTPUT :

30

3.14159

8. Explain what happens to `__name__`:

- When a Python file is executed directly
- When the same file is imported as a module

1) **When a Python file is executed directly**, the Python interpreter automatically sets `__name__` to `"__main__"`. As a result, the code inside `if __name__ == "__main__":` is executed.

2) **When the same file is imported as a module**, the interpreter sets `__name__` to the **module's name** (e.g., "program"). In this case, the condition `if __name__ == "__main__":` evaluates to **False**, so the code inside that block is not executed.

File 1: A7Q8.py

```
def add(a, b):  
    return a + b  
  
print("__name__ =", __name__)  
  
if __name__ == "__main__":  
    print("Calculator is running directly.")  
    print("10 + 20 =", add(10, 20))
```

File 2: A7Q8(1).py

```
import A7Q8  
  
print("Main program is running.")
```

OUTPUT :

`__name__ = A7Q8`

Main program is running.

Notice that these lines do not appear:

Calculator is running directly.

10 + 20 = 30

When main.py imports calculator, Python says: "calculator.py is **not** the main program."

So Python automatically sets `__name__ = "calculator"`

Now condition `if __name__ == "__main__":` become `if "calculator" == "__main__":` which is False.

Therefore, Python skips everything inside the if block.

This is why `if __name__ == "__main__":` is commonly used to keep **test code, examples, or the program's entry point** separate from reusable functions and classes.

9. Why do we write the condition: `if __name__ == "__main__":`

1. According to the official Python documentation, we write:

`if __name__ == "__main__":`

to ensure that a block of code **runs only when the file is executed directly**, and **does not run when the file is imported as a module**.

```
def add(a, b):  
    return a + b
```

```
def main():
    print(add(10, 20))

if __name__ == "__main__":
    main()
```

acts like a gate that says:

"Run this demonstration only when this book is opened directly, not when someone is just borrowing a recipe."

10. Explain the difference between:

- return x
- return x, y
- return

return x : Returns a single value One object (x)

```
def square(n):
    return n * n

def main():
    result = square(5)
    print(result)

if __name__ == "__main__":
    main()
```

OUTPUT :
25

This statement returns **multiple values**.

Important: Python actually returns **one tuple** containing multiple values.

return x, y : Returns multiple values (actually a tuple) A tuple (x, y)

```
def calculate(a, b):
    return a + b, a - b

result = calculate(10, 5)

print(result)
```

OUTPUT :
(15, 5)

return : Returns no explicit value None

```
def greet():  
    print("Welcome")  
    return
```

```
result = greet()
```

```
print(result)
```

OUTPUT :

None